

Fraud's Bargain Attack: Generating Adversarial Text Samples via Word Manipulation Process

Mingze Ni , Zhensu Sun , and Wei Liu , *Senior Member, IEEE*

Abstract—Recent research has revealed that natural language processing (NLP) models are vulnerable to adversarial examples. However, the current techniques for generating such examples rely on deterministic heuristic rules, which fail to produce optimal adversarial examples. In response, this study proposes a new method called the Fraud's Bargain Attack (FBA), which uses a randomization mechanism to expand the search space and produce high-quality adversarial examples with a higher probability of success. FBA uses the Metropolis-Hasting sampler, a type of Markov Chain Monte Carlo sampler, to improve the selection of adversarial examples from all candidates generated by a customized stochastic process called the Word Manipulation Process (WMP). The WMP method modifies individual words in a contextually-aware manner through insertion, removal, or substitution. Through extensive experiments, this study demonstrates that FBA outperforms other methods in terms of attack success rate, imperceptibility and sentence quality.

Index Terms—Adversarial learning, evasion attacks, natural language processing.

I. INTRODUCTION

AI MODELS that process text data have been widely used in various real-world scenarios, but they are surprisingly vulnerable. Attackers can significantly reduce the performance of these models by intentionally introducing maliciously designed typos, words, and letters into the input [1]. These modified inputs are referred to as adversarial examples. The growing prevalence of adversarial examples raises serious concerns about the comprehension and security of AI models.

A textual attack on an NLP model involves generating adversarial perturbations, which can be classified into character-level, word-level, and sentence-level attacks based on the attack strategy [2]. Character-level attacks involve manipulating characters to deceive the model, but this approach can result in misspelled words that can be easily detected and corrected by spell checkers [3]. Sentence-level attacks, on the other hand, involve inserting new sentences, paraphrasing sentence pieces, or changing the structure of the original sentences, but they often

produce incomprehensible sentences [4], [5]. Therefore, word-level attacks have received more attention from researchers because they are more effective and imperceptible than character-level and sentence-level attacks [2]. Attackers typically replace the original word with another heuristically chosen word, but existing methods focus mainly on substitutions, with limited exploration of insertion and removal of words [6], [7], [8], [9]. Furthermore, there is no prior work on using a combination of word substitution, insertion, and removal strategies to generate attacks, which limits the search space for adversarial examples and hinders attack performance. Most algorithms also use a word importance rank (WIR) to launch attacks based on a preset number of perturbed words (NPW) from the top of WIR. However, the optimal NPW varies with different target texts, making presetting challenging [8], [10]. Additionally, the different ways of defining WIR, such as probability-weighted word saliency (PWWS) ranking and ranks for drops of classifiers' predictions and transformer logits, make WIR-based methods ineffective in achieving imperceptibility, as they may change the semantics of the original sentences.

This study proposes a new attacking algorithm, called Fraud's Bargain attack (FBA), to tackle the limitations of existing word-level attacks. FBA utilizes a stochastic process called Word Manipulation Process (WMP), which considers word substitution, insertion, and removal strategies to generate potential adversarial candidates. The Metropolis-Hasting algorithm is used to select the best candidates based on a customized acceptance probability, which minimizes semantic deviation from the original sentences. WMP proposes malicious candidates that could be potential adversarial examples, while MH algorithm acts as an experienced 'fraud' to statistically evaluate their quality and select the best candidates.

FBA has two key advantages over most existing attacks in the literature. First, FBA offers a large search space for generating adversarial examples by allowing word manipulation through insertion, substitution, and deletion, unlike many attacks that rely solely on word substitutions. This increased search space allows for a theoretically higher probability of generating effective adversarial examples. Second, FBA employs a customized word distribution to stochastically select the attacked positions, enabling each word in the context to be chosen based on its importance. This approach allows for greater flexibility and adaptability compared to methods that rely on presetting the number of perturbed words (NPW). Additionally, FBA regulates the Metropolis-Hasting (MH) sampler based on the imperceptibility of the attacks and minimal deviation from the original

Manuscript received 2 March 2023; revised 10 October 2023; accepted 23 December 2023. Date of publication 9 January 2024; date of current version 10 June 2024. Recommended for acceptance by W. Zhao. (Corresponding author: Wei Liu.)

Mingze Ni and Wei Liu are with the School of Computer Science, University of Technology Sydney, Sydney, NSW 2007, Australia (e-mail: mingze.ni@student.uts.edu.au; wei.liu@uts.edu.au).

Zhensu Sun is with the School of Information Science and Technology, ShanghaiTech University, Shanghai 201210, China (e-mail: sunzhs@shanghaitech.edu.cn).

Digital Object Identifier 10.1109/TKDE.2024.3349708

semantics, resulting in an adaptive setting of NPW that can vary depending on the input text. This approach allows for more successful attacks, as the optimal NPW can differ depending on the characteristics of the input text.

In summary, our work addresses the challenge of generating effective and semantically preserving adversarial examples for natural language processing tasks, and the main contributions in this work are as follows:

- We develop a probabilistic approach called the Word Manipulation Process (WMP) to generate a wide range of potential adversarial examples. The WMP involves the manipulation of words through operations such as insertion, removal, and substitution, which provides a large search space for generating adversarial candidates.
- Our paper introduces a powerful adversarial attack method called Fraud's Bargain Attack (FBA) that uses the Metropolis-Hasting (MH) algorithm to construct an acceptance probability. This probability is then applied to select the highest-quality adversarial candidates generated by the Word Manipulation Process (WMP). By using the acceptance probability, our attack method can avoid getting stuck in local optima and can generate solutions that are closer to the global optima.
- In order to demonstrate the effectiveness of our proposed Fraud's Bargain Attack (FBA) method, we conducted experiments on real-world public datasets. Our experimental results indicate that our method outperformed other state-of-the-art methods in terms of attack success rate, preservation of semantics, and overall sentence quality. These findings provide strong evidence to support the effectiveness of our proposed method in generating high-quality adversarial examples while preserving the original semantics of the input sentence.

The rest of this paper is structured as follows. We first review adversarial attacks for NLP models and the applications of Markov Chain Monte Carlo (MCMC) methods to NLP models in Section II. We detail our proposed method in Sections III and IV and then evaluate the performance of the proposed method through empirical analysis in Section V. We conclude the paper with suggestions for future work in Section VI.

II. RELATED WORK

In this section, we provide a review of the literature on word-level textual attacks and MCMC sampling in NLP.

A. Word-Level Attacks to Classifiers

Word-level attacks pose non-trivial threats to NLP models by locating important words and manipulating them for targeted or untargeted purposes. With the help of an adopted fast gradient sign method (FGSM) [11], Papernot [12] was the first to generate word-level adversarial examples for classifiers. Notably, while textual data is naturally discrete and more perceptible than image data, many gradient-based textual attacking methods inherited from computer vision are not effective enough, making textual attack a challenging problem to address. Alzantot [13] transferred the domain of adversarial attacks to an optimization problem by formulating a customized objective function.

With genetic optimization, they succeeded in generating adversarial examples by sampling the qualified genetic generations that break out of the encirclement of the semantic threshold. However, genetic algorithms can be inefficient. Since the word embedding space is sparse, performing natural selection for languages in such a space can be computationally expensive. Jia [14] proposed a faster version of Alzantot's adversarial attacks by shrinking the search space, which accelerates the process of evolving in genetic optimization. Although Jia has greatly reduced the computational expense of genetic-based optimization algorithms, searching or optimizing algorithms in word embedding spaces, such as GloVe [15] and Word2Vec [16], are still not efficient enough.

To ease the searching process, embedding-based algorithms often resort to using a counter-fitting method to post-process attacker's vectors to ensure the semantic constraint [17]. Compared with word embedding methods, utilizing a well-organized linguistic thesaurus, e.g., synonym-based WordNet [18] and sememe-based HowNet [19], offers a simpler and more straightforward implementation. Ren [10] sought synonyms based on WordNet synsets and ranked word replacement order via probability-weighted word saliency (PWWS). Zang [7] and Yang [1] both demonstrated that the sememe-based HowNet can provide more substitute words using the Particle Swarm Optimization (PSO) and an adaptive monotonic heuristic search to determine which group of words should be attacked. Additionally, some recent studies utilized a masked language model (MLM), such as BERT [20] and RoBERTa [21], to generate contextual perturbations [6], [8]. The pre-trained MLM ensures that the predicted token fits the sentence's grammar correctly but might not preserve the semantics.

B. Markov Chain Monte Carlo in NLP

Markov Chain Monte Carlo (MCMC) stands as a versatile technique for approximating samples from complex distributions, finding critical applications in areas like optimization [22], machine learning [23], and quantum simulation [24]. Among the myriad MCMC algorithms, the Metropolis-Hastings (MH) algorithm is arguably the most foundational. Tracing its roots to the pioneering work of Metropolis et al. [25], and later extended by Hastings [26], the MH algorithm emerges as an indispensable tool in the MCMC arsenal. Its primary utility lies in generating a sequence of samples from a target distribution, especially when direct sampling poses challenges. The algorithm achieves this by formulating a Markov chain whose equilibrium distribution coincides with the desired distribution. At its core, the MH algorithm utilizes a proposal distribution to propose new candidate samples and employs an acceptance criterion to assure convergence to the intended distribution. While the MH algorithm's adaptability makes it suitable for a broad spectrum of applications, its efficiency hinges on the meticulous design of the proposal distribution. In the field of Natural Language Processing (NLP), MCMC methods, including MH, have been instrumental in tasks such as text generation, sentiment analysis, and named entity recognition, aiding in the evolution of models that interpret and comprehend human language.

Algorithm 1: Word Manipulation Process.

Input: Number of iterations: T , Input text: x
Output: A set of adversarial candidates: $[x'_1, \dots, x'_T]$

```

1 candidates=[ ]
2  $x'_0 = x$  for  $t$  in  $T$  do
3   Given  $x'_{t-1}$ , sample  $e$  with Eq. 2
4   Given  $x'_{t-1}$ ,  $e$ , sample  $l$  with Eq. 3
5   Given  $x'_{t-1}$ ,  $e$ ,  $l$ , sample  $o$  with Eq. 10
6   Craft  $x'_t$  by taking action  $e, l, o$  to  $x'_{t-1}$ 
7   candidate.append( $x'_t$ )
8 return candidates

```

For text generation, Kumagai [27] propose a probabilistic text generation model which generates human-like text by inputting semantic syntax and some situational content. Since human-like text requests grammatically correct word alignment, they employed Monte Carlo Tree Search to optimize the structure of the generated text. In addition, Harrison [28] presents the application of MCMC for generating a story, in which a summary of movies is produced by applying recurrent neural networks (RNNs) to summarize events and directing the MCMC search toward creating stories that satisfy genre expectations. For sentimental analysis, Kang [29] applies the Gibbs sampler to the Bayesian network, a network of connected hidden neurons under prior beliefs, to extract the latent emotions. Specifically, they apply the Hidden Markov models to a hierarchical Bayesian network and embed the emotions variables as the latent variable of the Hidden Markov model. To accelerate the training speed of name entity recognition models, Sameer [30] proposes an adaptive MCMC sampling scheme for a graphical NLP model. They demonstrate that the proposed sampler can sample a set of latent variables which are asymptotic to the original transition distribution. The proposed sampler converges more quickly than gradient-based optimization from their experiments, and the sampled latent variables can represent the critical name entities.

III. WORD MANIPULATION PROCESS

In this section, we provide a detailed explanation of the Word Manipulation Process (WMP), which is used to generate adversarial candidates. The method for selecting the generated candidates will be described in the following section. Let $D = (x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$ be a dataset consisting of m samples, where x and y represent the input text and its corresponding class, respectively. The victim classifier F maps from the text space $\mathcal{X}$ to the class space using a categorical distribution, such that $F(\cdot) : \mathcal{X} \rightarrow (0, 1)^K$, where K is the number of classes. Given an input text x consisting of n words $w_1, w_2, \dots, w_n$, we refer to a modified version of the text as an adversarial candidate and denote it as x' . The final chosen adversarial example is referred to as x^* .

The Word Manipulation Process (WMP) takes three steps to modify the current text state x . First, an action e is randomly selected from the set of possible actions, which includes inserting a word at the beginning of the sentence, substituting a word, or removing a word. Second, the position l where the manipulation

should take place is determined by drawing from a customized categorical distribution. Third, if word insertion or substitution is selected, WMP generates a candidate by combining synonyms of the original words and using a pre-trained masked language model (MLM) such as BERT [20], XML [31] and MPNet [32]. The algorithm for WMP is presented in Algorithm 1. In the following, we elaborate on the details of the three steps of WMP:

A. Action

We draw $e \in \{0, 1, 2\}$ from a categorical distribution

$$p(e|x) = \begin{cases} \mathbf{P}_{ins} & e = 0, \\ \mathbf{P}_{sub} & e = 1, \\ \mathbf{P}_{rem} & e = 2 \end{cases}, \quad (1)$$

where $\mathbf{P}_{ins} + \mathbf{P}_{sub} + \mathbf{P}_{rem} = 1$, and $\mathbf{P}_{ins}$, $\mathbf{P}_{sub}$, and $\mathbf{P}_{rem}$ represent probability of insertion (0), substitution (1) and removing (2), respectively. The probabilities of these types of attacks can be set by the attacker's preference.

B. Position

Given a certain action e , we need to select one target word at location l in the sentence to implement the attack. Considering the effectiveness of the selection, higher probabilities should be assigned to the words with more influence. To solve this, we use the changes of victim classifiers' logits, $I = [I_{w_1}, \dots, I_{w_i}, \dots, I_{w_n}]$, before and after deleting a word. Such a drop of logits for i th word, I_{w_i} , is mathematically formulated as

$$I_{w_i} = F_{y, \logit}(x) - F_{y, \logit}(x_{w_i}), \quad (2)$$

where $F_{y, \logit}(\cdot)$ is the classifier returning the logit of the correct class, and $x_{w_i} = [w_1, \dots, w_{i-1}, w_{i+1}, \dots, w_n]$ is the text with w_i removed. Different from word importance rank (WIR), we utilize drops of logits I to craft categorical distribution on position l , $p(l|e, x)$, by putting I to a softmax function as follows:

$$p(l|e, x) = \text{softmax}(I). \quad (3)$$

This way, locations of words (tokens) are assigned probabilities according to the words' influence on the classifier.

C. Word Candidates

Different actions require different search strategies for word candidates. To find the word candidates for substitution and insertion, we utilize an MLM and synonyms of the original words (calculated by nearest neighbours using the L2-norm of word embeddings), for parsing fluency and semantic preservation, respectively. As for word removals, we design a hesitation mechanism to maintain a probabilistic balance with word insertion. The details are demonstrated in the following paragraphs.

1) *Candidates for Substitution Attacks:* We mask the word on the selected position to construct

$$x_{sub}^* = [w_1, \dots, [MASK], \dots, w_n],$$

and feed this masked sentence x_{sub}^* into a MLM $\mathcal{M}(\cdot) : \mathcal{X} \rightarrow (0, 1)^d$ to obtain a distribution about word candidates o across

all the words in the dictionary size d . The distribution is below

$$p_{\mathcal{M}}(o|l, e, x) = \mathcal{M}(x_{sub}^*). \quad (4)$$

The MLM relies on softmax to output a distribution on the dictionary, but most words from the dictionary can be grammatically improper, and the probability of selecting one of these words can be high. To this end, we tend to create another distribution with respect to the k top word candidates from the MLM. By mixing such a distribution with the MLM distribution, the probability of selecting grammatically improper words can be decreased. To construct such a k top words distribution, we choose the k top words $G_{\mathcal{M}}^{sub} = \{w_{(\mathcal{M},1)}^{sub}, \dots, w_{(\mathcal{M},k)}^{sub}\}$ from $\mathcal{M}$ and treat every word from this set equally important

$$p_{\mathcal{M}}^{top}(o|l, e, x) = \mathbb{1}(o \in G_{\mathcal{M}}^{sub}) \frac{1}{k}, \quad (5)$$

where $\mathbb{1}(\cdot)$ is an indicator function. In such a setting, the top candidates from MLM are attached with more importance.

Synonyms often fit more seamlessly with parsers in MLM, and they generally achieve a higher probability in MLM, as evidenced by studies such as [33], [34]. While MLM has already demonstrated its prowess in maintaining semantics, we emphasize a robust selection of synonyms based on word embeddings. This approach to synonym selection has been corroborated as effective by research like [13], [14], [35]. Specifically, we perform synonym extraction by gathering a word candidate set for top k replacements of the selected word. Specifically, we use L-2 norm as the metric to perform kNN inside word embedding space from BERT and construct such a synonym candidates set $G_{nn} = \{w_{(\mathcal{M},1)}, \dots, w_{(\mathcal{M},k)}\}$, with top- k nearest neighbors from the embedded spaces as synonyms for the word on selected position l . In this way, we construct the synonym words distribution as follows:

$$p_{nn}(o|l, e, x) = \mathbb{1}(o \in G_{nn}) \frac{1}{k}. \quad (6)$$

As we tend to generate parsing-fluent and semantic-preserving adversarial candidates, we combine the distributions in (4), (5) and (6) to construct a mixture distribution as the final distribution to draw the substitution

$$p_{sub}(o|e, l, x) = a_1 p_{\mathcal{M}}(o|l, e, x) + a_2 p_{\mathcal{M}}^{top}(o|l, e, x) + a_3 p_{nn}(o|l, e, x), \quad (7)$$

where $a_1 + a_2 + a_3 = 1$, $a_1, a_2, a_3 \in (0, 1)$, while a_1 , a_2 and a_3 are hyper-parameters for weighing the corresponding distribution.

2) *Candidates for Insertion Attacks*: Searching word candidates for insertion attacks follows a similar logic as substitutions but without a synonym search. We construct the mask sentence $x_{ins}^* = [w_1, \dots, w_{l-1}, [MASK], w_l, \dots, w_n]$ by inserting a masked token on the left side of a selected position, then apply the MLM $\mathcal{M}$ to the mask sentence for extracting the output of the softmax layer, $\mathcal{M}(x_{ins}^*)$. Following the same logic as (5), we select the top k word candidates $G_{\mathcal{M}}^{ins} = \{w_{(\mathcal{M},1)}, \dots, w_{(\mathcal{M},k)}\}$. Similar to (7), insertion word candidate distribution can be

constructed as follows:

$$p_{ins}(o|l, e, x) = b_1 \mathcal{M}(x_{ins}) + b_2 \mathbb{1}(o \in G_{\mathcal{M}}^{ins}) \frac{1}{k}, \quad (8)$$

where $b_1 + b_2 = 1$, $b_1, b_2 \in (0, 1)$, b_1 and b_2 are hyper-parameters for weighing the corresponding distribution.

3) *Candidates for Removal Attacks*: Since word candidates for insertion and substitution can be drawn from a significantly large dictionary, these two actions will provide a large variance of adversarial candidates. Differently, removal does not require a selection of word candidates but directly removes the word on the selected position, which will lead to a low variety of adversarial candidates crafted by removal. The consequence of such a low variety is that the probability of crafting the same adversarial candidate is much higher than inserting and substituting. To balance such a probability, we design a Bernoulli distribution for determining word removals. Specifically, we craft the removal word candidates set $G^{rem} = \{0, 1\}$, where 0 and 1 represent remaining and removing the selected word, respectively. The distribution is as follows:

$$p_{rem}(o|e, l, x) = \begin{cases} 1 - \frac{1}{k} & o = 0, \\ \frac{1}{k} & o = 1 \end{cases}. \quad (9)$$

With the above distribution, the $o = 1$ (i.e., to remove the word) is selected to replace the original word with probability $\frac{1}{k}$. The rationale of using $\frac{1}{k}$ is to decrease the probability of repeatedly proposing the same perturbed sentence with action removal such that it is approximate to the probability of word replacement and insertion as in (4) and (8): while each replacement word has a probability of $\frac{1}{k}$ for being chosen, the removal, $o = 1$, has the same probability of being selected in removal attacks.

With the three WMP steps, we summarize the probability density function for word candidates

$$p(o|e, l, x) = \begin{cases} p_{ins}(o|e, l, x) & e = 0, \\ p_{sub}(o|e, l, x) & e = 1, \\ p_{rem}(o|e, l, x) & e = 2 \end{cases}. \quad (10)$$

D. Integration of the Three WMP Steps

By iteratively running WMP T times with an initial start at original input text ($x'_0 = x$), we can get a sequence of adversarial candidates $x'_T = [x'_1, \dots, x'_t, \dots, x'_T]$. By applying the Bayes rule, we can derive the WMP's distribution from the iteration t to $t + 1$ as the following equation:

$$\begin{aligned} \text{WMP}(x'_{t+1}|x'_t) &= p(e, l, o|x'_t) \\ &= p(e|x'_t)p(l|e, x'_t)p(o|e, l, x'_t). \end{aligned} \quad (11)$$

Two Theoretical Merits of WMP

WMP is expected to own two major merits: enlarging the searching domain and the ability to correct possible wrong manipulation. These two merits can be guaranteed by the aperiodicity shown in the following theorem.

Theorem 1: Word Manipulation Process (WMP) is aperiodic.

Proof: Suppose we have two arbitrary text samples $x_i, x_j \in \mathcal{X}$ from text space $\mathcal{X}$. $x_i = [w_1^i, \dots, w_{n_i}^i]$ $x_j = [w_1^j, \dots, w_{n_j}^j]$ have n_i and n_j words, respectively. To prove the process is

aperiodic by definition, we need to show

$$\exists N \leq \infty, \mathbb{P}(x^{(N)} = x_j | x_i) > 0, \quad (12)$$

which means that there always exists $\exists N \leq \infty$ that can make the probability of transferring x_i to x_j , after N steps, greater than zero.

Because a text dataset is discrete and WMP is time-discrete, WMP is a Markovian process. Therefore, we apply the Chapman–Kolmogorov equation [36], to derive the following equation:

$$\begin{aligned} \mathbb{P}(x^{(N)} = x_j | x_i) &= \sum_{x^{(t)} \in \mathcal{X}} \text{WMP}(x^{(1)} | x_i) \\ &\quad \dots \text{WMP}(x^{(t)} | x^{(t-1)}) \dots \text{WMP}(x_j | x^{(N-1)}), \end{aligned} \quad (13)$$

where $\text{WMP}(\cdot)$ denotes the Word Manipulation process (WMP) and $x^{(t)}$ denotes the arbitrary text in text space $\mathcal{X}$ after running WMP for t times. We try to prove aperiodicity with a special case, let $N = n_i + n_j$. $\mathcal{A}$: first inserting all the words from x_j to the x_i , then remove all words from x_i . This process can be illustrated as follows:

$$\begin{aligned} \mathcal{A}: \quad x_i &\xrightarrow[n_j \text{ times insert}]{n_j \text{ times}} x^{n_j} = [w_1^i, \dots, w_{n_i}^i, w_1^j, \dots, w_{n_j}^j] \\ &\xrightarrow[n_j \text{ times remove}]{n_j \text{ times}} x_j = [w_1^j, \dots, w_{n_j}^j]. \end{aligned}$$

As $\mathcal{A}$ is the special case of the N times iterations, we have

$$\mathbb{P}(\mathcal{A}) \leq \mathbb{P}(x^{(N)} = x_j | x_i). \quad (14)$$

Moreover, the WMP inserts one word on any position based on softmax, which outputs non-zero probabilities, therefore we can derive

$$0 < \mathbb{P}(\mathcal{A}) \leq \mathbb{P}(x^{(N)} = x_j | x_i). \quad (15)$$

Therefore, we find that for arbitrary $x_i, x_j \in \mathcal{X}$, there always exist $N = n_i + n_j \leq \infty$ that can make the probability of transfer x_i to x_j after N time larger than zero, i.e., $\mathbb{P}(x^{(N)} = x_j | x_i) > 0$. According to the definition, we have successfully proved Theorem 1.

Aperiodicity implies that, given a large enough number of iterations T , an arbitrary x can be perturbed to any text x' in text space, i.e., $\text{WMP}(x | x') \neq 0$. The first merit of aperiodicity, WMP theoretically guarantees that the searching domain is enlarged to generate the most effective adversarial candidates. Since WMP samples the adversarial candidates with randomness, there is a tiny possibility of crafting a bad adversarial candidate. With aperiodicity, WMP is eligible for correcting this bad manipulation by reversing the bad sample x'_{t+1} to the previous state x'_t .

IV. ADVERSARIAL CANDIDATE SELECTION BY METROPOLIS-HASTING SAMPLING

A simple approach to selecting adversarial examples is to test all candidates generated by WMP and choose the best-performing one. However, this method can be time-consuming

Algorithm 2: Fraud's Bargain Attack (FBA).

Input: Input text: x , Number of Sample: T

Output: An adversarial example

```

1  $Adv\_set = [ ]$ 
2  $x_1 = x$ 
3 for  $t$  in  $range(T)$  do
4   Sample  $x_t$  from WMP given  $x_{t-1}$  with Eq. 11
5   Sample  $u$  from Uniform distribution,
   Uniform(0,1)
6   Calculate the acceptance probability,
    $prob = \alpha(x_{t+1}, x_t)$  with Eq. 22
7   if  $u < prob$  then
8      $x_t = x_{t+1}$ 
9      $Adv\_set.append(x_{t+1})$ 
10  else
11     $x_t = x_t$ 
12     $Adv\_set.append(x_t)$ 
13  return  $Adv\_set$ 
14 Choose the candidate with the least modification as
   the adversarial example  $x^*$ .
15 return adversarial example  $x^*$ 

```

and may result in over-modified adversarial examples. To address this issue, we propose the Fraud's Bargain Attack (FBA), which enhances WMP using the Metropolis-Hastings (MH) algorithm. FBA selects adversarial candidates by evaluating them using a customized adversarial distribution.

A. Markov Chain Monte Carlo

Markov chain Monte Carlo (MCMC), a statistically generic method for approximate sampling from an arbitrary distribution, can be applied in a variety of fields. The main idea is to generate a Markov chain whose equilibrium distribution is equal to the target distribution [37]. Metropolis Hasting (MH) sampler [25], from which MCMC methods originate, applies the following setting: suppose that we wish to generate samples from an arbitrary multidimensional probability density function (PDF)

$$f(\mathbf{x}) = \frac{p(\mathbf{x})}{\mathcal{Z}}, \quad (16)$$

where $p(\mathbf{x})$ is a known positive function and $\mathcal{Z}$ is a known or unknown normalizing constant. Let $q(\mathbf{y} | \mathbf{x})$ be a proposal or instrumental density: a Markov transition density describing how to go from state $\mathbf{x}$ to $\mathbf{y}$. The MH algorithm is based on the following “trial-and-error” strategy by defining an accept probability $\alpha(\mathbf{y} | \mathbf{x})$ as following:

$$\alpha(\mathbf{y} | \mathbf{x}) = \min \left\{ \frac{f(\mathbf{y})q(\mathbf{x} | \mathbf{y})}{f(\mathbf{x})q(\mathbf{y} | \mathbf{x})}, 1 \right\}, \quad (17)$$

to decide whether the new state $\mathbf{y}$ is accepted or rejected. To be more specific, we sample a random variable u from a Uniform distribution ranging 0 to 1, $u \sim \text{Unif}(0, 1)$, and if $u < \alpha(\mathbf{x}, \mathbf{y})$ then $\mathbf{y}$ is accepted as the new state, otherwise the chain remains at $\mathbf{x}$. The fact that equilibrium distribution of MH Markov Chain is

equal to the target distribution is guaranteed by the local/detailed balance equation.

A good property of MH sampler is that in order to evaluate the accept rate $\alpha(\mathbf{x}, \mathbf{y})$, it is only necessary to know the positive function $q(\mathbf{y} | \mathbf{x})$, which is also known as kernel density. In addition, the efficiency of the MH sampler depends on the choice of the transition density function $q(\mathbf{y} | \mathbf{x})$. Ideally, $q(\mathbf{y} | \mathbf{x})$ should be "close" to $f(\mathbf{y})$, with respect to $\mathbf{x}$.

B. Adversarial Distribution

We argue that adversarial examples should work with imperceptible manipulations to input text. Therefore, given text x , we construct the adversarial target distribution $\pi(x') : \mathcal{X} \rightarrow (0, 1)$ to measure the classifier's deprivation once under attack with a heavy penalty on change of semantics. Concretely, we measure the classifier's deprivation by defining a measure of distance to perfection, R , based on the confidence of making wrong predictions $1 - F_y(x')$, where $F_y : \mathcal{X} \rightarrow [0, 1]$ is the confidence of predicting correct class. The higher the value of the distance to perfection R , the more successful the attack. Meantime, we add a regularizer on semantic similarity, $\text{Sem}(\cdot)$. Thus the mathematical formulation is as follows:

$$\pi(x'|x, \lambda) = \frac{R + \lambda \text{Sem}(x', x)}{C} \quad (18)$$

$$R = \begin{cases} 1 - F_y(x') & F_y(x') > \frac{1}{K}, \\ 1 - \frac{1}{K} & F_y(x') \leq \frac{1}{K} \end{cases} \quad (19)$$

$$C = \sum_{x' \in \mathcal{X}} R + \lambda \text{Sem}(x', x), \quad (20)$$

where K is the number of classes. In (18), we construct adversarial distribution by utilizing a hyper-parameter λ to combine the attack performance R and semantic similarity $\text{Sem}(\cdot)$. In (20), C represents the constant normalizing $\pi(x')$ to ensure the distribution condition, $\sum_{x' \in \mathcal{X}} \pi(x'|x, \lambda) = 1$. To keep more semantics, we let $\text{Sem}(x', x)$ denote the semantic similarity between adversarial example x' and original text x . In general, the $\text{Sem}(\cdot)$ is implemented with the cosine similarity between sentence encodings from a pre-trained sentence encoder, such as USE [38].

In spite of the use of the semantic regularizer, we argue that a high R might still cause a thrilling semantic loss because the value of $\pi(x'|x)$ might go up with large increases of R and small drops of semantic similarity $\text{Sem}(x', x)$. Thus, for a further improvement on semantic preservation, we let the R be associated with a cut-off value at $\frac{1}{K}$ when the class is successfully misclassified (i.e., when $F_y(x') \leq \frac{1}{K}$). Note that when $F_y(x') \leq \frac{1}{K}$, the classifier will misclassify x' to one of the other $K - 1$ classess other than y . By having the mechanism of setting R to $\frac{1}{K}$ whenever misclassification is achieved, all successful adversarial examples will have the same R value. This way, their optimization with π will then focus on maximizing their semantic similarity $\text{Sem}(x', x)$ with the original texts.

C. Fraud's Bargain Attack via Metropolis Hasting Sampler

Metropolis Hasting simulates a target distribution by using a proposing function to offer a trial state which is then accepted or rejected according to a customized acceptance probability. Specifically, given a target distribution $Q(\cdot)$, the MH sampler utilizes a proposing function $q(s_{t+1}|s_t)$ (transition density from s_t to s_{t+1}) to construct a Markov Chain, whose equilibrium distribution is our target distribution. By this probabilistic mechanism, the proposing function would propose a trial state s_{t+1} given the current state s_t and the acceptance probability $\alpha(s_{t+1}|s_t)$, as shown in (21)

$$\alpha(s_i, s_{i+1}) = \min \left(1, \frac{Q(s_{i+1})}{Q(s_i)} \frac{q(s_i | s_{i+1})}{q(s_{i+1} | s_i)} \right). \quad (21)$$

Based on such a setting, we construct FBA by considering the adversarial distribution (18) and the WMP as the MH's target distribution and proposing function, respectively. In each iteration of FBA, we use WMP to propose a trial state x_{t+1} and calculate the acceptance probability $\alpha(x_{t+1}|x_t)$. FBA's acceptance probability in (21) can then be mathematically formulated by using WMP as follows:

$$\alpha(x_{t+1}|x_t) = \min \left(1, \frac{\pi(x_{t+1})}{\pi(x_t)} \frac{\text{WMP}(x_t | x_{t+1})}{\text{WMP}(x_{t+1} | x_t)} \right), \quad (22)$$

where $\text{WMP}(x_t | x_{t+1})$ is guaranteed above zero by Theorem 1, and calculated by reversing the WMP process: removing the inserted word, inserting the removed word and recovering substituted word. Once the value of $\alpha(x_{t+1}|x_t)$ is computed, we proceed to sample u from a uniform distribution ranging from 0 to 1, i.e., $u \sim \text{Unif}(0, 1)$. If $u < \alpha(x_{t+1}|x_t)$ we will accept x_{t+1} as the new state, otherwise the state will remain as x_t . After performing T iterations, FBA produces a collection of adversarial candidates, among which we select the one with the smallest modification that successfully flips the predicted class. The complete process of FBA is presented in Algorithm 2. With this acceptance probability, we expect the algorithm to asymptotically converge to a candidate that compromises the model's performance while preserving sufficient semantic integrity.

V. EXPERIMENTS AND ANALYSIS

We assess the performance of our methods using popular and publicly accessible datasets, along with high-performing victim classifiers. In order to facilitate reproducibility, we have made our code and data available on a GitHub repository¹

A. Datasets and Victim Models

1) *Dataset*: We conduct experiments on four publicly accessible benchmark datasets. AG's News [39] is a news classification dataset with 127,600 samples belonging to 4 topic classes. Emotion [40] is a dataset with 20,000 samples and 6 classes. SST2 [41] is a binary class topic dataset with 9,613 samples. IMDB [41] is a binary class topic dataset with 50,000 labeled samples. Details of these datasets can be found in Table I.

¹<https://github.com/MingzeLucasNi/FBA>.

TABLE I
DATASETS AND ACCURACY OF VICTIM MODELS BEFORE ATTACKS

Dataset	Size	Task	Model	Accuracy
AG's News	127000	News topics	BERT-C TextCNN	94% 90%
Emotion	20000	Sentiment analysis	BERT-C TextCNN	97% 93%
SST2	9613	Sentiment analysis	BERT-C TextCNN	91% 83%
IMDB	50000	Movie review	BERT-C TextCNN	93% 88%

2) *Victim Models*: We apply our attacks to two popular and well-performed victim models below.

2) *BERT-Based Classifiers*: To perform experiments, three BERT-based models, collectively called BERT-C models, were selected and pre-trained by Huggingface. The structures of the classifiers were adjusted based on the dataset sizes. BERT classifiers were trained on AG's News and Emotion datasets using Distil-RoBERTa-base and Distil-BERT-base-uncased, respectively. The smaller version of BERT, BERT-base-uncased, was used for the SST2 dataset. The accuracy of the models before the attack can be found in Table I, and the models are publicly available for reproducibility purposes.

2) *TextCNN-Based Models*: The TextCNN victim model is a different type of model from the BERT-based models. It has a 100-dimension embedding layer followed by a 128-unit long short-term memory layer. This classifier is trained 10 epochs by ADAM optimizer with the default hyper-parameter in Pytorch. The accuracy of these TextCNN-base models is also shown in Table I.

B. Baselines

We choose the following solid baselines:

- Faster Alzantot Genetic Algorithm (FAGA) [14] accelerate Alzantot Genetic Algorithm [13], by bounding the searching domain of genetic optimization.
- BAE [6] replaces and inserts tokens in the original text by masking a portion of the text and leveraging the BERT-MLM.
- BERT-Attack [8] takes advantage of BERT MLM to generate candidates and attack words by the static WIR descending order.
- A2T [43] uses a gradient-based word importance ranking method to iteratively replace each word with synonyms generated from a counter-fitted word embedding.
- CLARE [33] applies a sequence of contextualized perturbation actions to the input. Each can be seen as a local mask-then-infill procedure: it first applies a mask to the input around a given position and then fills it in using a pre-trained MLM.
- PWWS [10] method proposed in Ren et al. (2019) selects candidate words from WordNet, a large lexical database of English, and then sorts them in attack order by multiplying the word saliency with the probability variation.

- Particle Swarm Optimization (PSO) [7] is a method that selects word candidates from HowNet [19] and uses PSO to find adversarial text via combinatorial optimization.

C. Evaluation Metrics and Experimental Setting

We use the following six metrics to measure the performance of adversarial attacks.

- Successful attack rate (SAR) denotes the percentage of adversarial examples that can successfully attack the victim model.
- Universal Sentence Encoder (USE) [38] measures the semantic similarity by calculating the cosine similarity between the input and its adversary.
- Modification Rate (Mod) is the percentage of modified tokens. Each replacement, insertion or removal action accounts for one modified token.
- ROUGE: measures the overlap of n-grams between the candidate and reference sentences, to evaluate the alignment similarity between adversarial examples and original sentences.
- Grammar Error Rate (GErr) is measured by the absolute rate of increased grammatical errors in the successful adversarial examples, compared to the original text, where we use LanguageTool [44] to obtain the number of grammatical errors.
- Perplexity (PPL) denotes a metric used to evaluate the fluency of adversarial examples and is broadly applied in the literature [7], [33].

The SAR is tailored for assessing attack performance, focusing on the imperceptibility of adversarial examples via semantic similarity, modification extent, and word order. We utilize USE for semantic analysis, Mod to quantify modifications, and ROUGE to evaluate syntactical structure. Fluency is measured using GErr for grammatical accuracy and PPL for readability, ensuring the adversarial examples are seamless and indistinguishable.

For settings of FBA, we set WMP action proposing probability in (1) as $P_{ins} = 0.2$, $P_{sub} = 0.6$, and $P_{rem} = 0.2$. While setting up the distribution for selecting the substitution and insertion words, we believe MLM and Synonyms are equally important. Therefore we set the weights of these two methods equal by setting $a_1 = 0.1$, $a_2 = 0.4$, $a_3 = 0.5$ and $b_1 = 0.5$, $b_2 = 0.5$ from (7) and (8), respectively. In addition, we only consider examples with a USE greater than 0.5 and where the modification rate remains below 0.25. We believe that an attack which modifies too many words or significantly alters the semantics, even if successful, cannot be deemed a quality attack

D. Experimental Results and Analysis

The experimental results of SAR and the imperceptibility performance (ROUGE, USE) and sentence quality (GErr, PPL) are listed in Tables II, III, and IV, respectively. To give an intuitive of the generated examples, we also show two generated adversarial examples in Tables V and VI. We manifest the three contributions mentioned in the Introduction section by asking four research questions:

TABLE II
SUCCESSFUL ATTACK RATE (SAR) OF ATTACK ALGORITHMS

Dataset	Model	Attack Methods								
		WMP	A2T	BAE	FAGA	BERT.A	CLARE	PWWS	PSO	FBA
AG's News	BERT-C	21.39%	15.25%	19.50%	25.17%	47.71%	71.32%	<u>76.75%</u>	67.76%	81.90%
	TextCNN	25.19%	33.31%	39.13%	56.10%	70.21%	79.31%	<u>85.31%</u>	76.24%	93.12%
Emotion	BERT-C	44.91%	48.06%	62.68%	78.21%	85.90%	<u>99.06%</u>	91.75%	94.76%	99.15%
	TextCNN	83.11%	81.11%	85.34%	90.10%	95.11%	97.11%	<u>98.20%</u>	<u>99.00%</u>	100%
SST2	BERT-C	52.43%	41.74%	55.14%	77.07%	77.21%	94.79%	93.9%	<u>96.62%</u>	99.31%
	TextCNN	71.69%	77.41%	80.15%	92.14%	83.23%	85.55%	<u>98.13%</u>	<u>92.20%</u>	100%
IMDB	BERT-C	79.1%	76.1%	80.3%	90.4%	89.1%	95.1%	94.1%	100.0%	100.0%
	TextCNN	77.0%	81.0%	89.3%	92.5%	91.1%	98.4%	99.6%	100.0%	100.0%

The higher values of SAR indicate better performance. For each row, the highest SAR is highlighted in bold, the second highest SAR is highlighted by underline, and the third highest SAR is denoted with italic.

TABLE III
IMPERCEPTIBILITY PERFORMANCE (ROUGE, USE, MOD) OF ATTACK ALGORITHMS

Dataset	Model	Metrics	Attack Methods								
			WMP	A2T	BAE	FAGA	BERT.A	CLARE	PWWS	PSO	FBA
AG's News	BERT-C	ROUGE ↑	0.5149	0.8311	0.8145	0.7721	0.8003	0.6673	<i>0.8356</i>	0.8521	<u>0.8453</u>
		USE ↑	0.7083	0.7143	0.7234	<i>0.7534</i>	0.7334	0.6603	0.7332	<u>0.7732</u>	0.8001
		Mod ↓	17.2%	13.3%	<u>11.3%</u>	14.6%	17.4%	14.2%	17.9%	21.6%	11.0%
	TextCNN	ROUGE ↑	0.5149	0.8291	0.8355	0.7663	<u>0.8643</u>	0.6969	0.8366	<i>0.8401</i>	0.8711
		USE ↑	0.7083	0.7542	0.7257	0.8014	<i>0.8103</i>	0.7121	<u>0.8111</u>	<i>0.8103</i>	0.8224
		Mod ↓	19.2%	11.9%	10.3%	11.5%	15.4%	14.1%	16.5%	15.5%	10.3%
Emotion	BERT-C	ROUGE ↑	0.5021	0.6111	0.5991	0.5911	<u>0.6401</u>	0.4441	0.6141	<i>0.6256</i>	0.6410
		USE ↑	0.8803	0.8951	0.8831	0.8940	<i>0.8714</i>	0.8201	<i>0.9012</i>	<u>0.9210</u>	0.9246
		Mod ↓	9.9%	9.1%	<u>7.7%</u>	9.8%	9.2%	11.2%	10.2%	12.0%	7.3%
	TextCNN	ROUGE ↑	0.6210	0.6891	0.6413	<i>0.6932</i>	0.6201	0.5623	<u>0.6994</u>	0.6613	0.7304
		USE ↑	0.7982	<i>0.8530</i>	0.7341	0.80412	0.8315	0.5631	<u>0.8920</u>	0.8342	0.9046
		Mod ↓	13.0%	10.3%	9.8%	<u>8.3%</u>	9.3%	15.3%	19.0%	18.0%	8.0%
SST2	BERT-C	ROUGE ↑	0.6621	0.6642	<u>0.7521</u>	<i>0.7201</i>	0.6352	0.5501	0.6341	0.7041	0.7540
		USE ↑	0.7938	0.8421	0.7304	0.7310	0.8564	0.7434	0.8422	0.8104	0.8820
		Mod ↓	18.1%	13.4%	<u>11.3%</u>	17.2%	13.5%	22.1%	17.2%	17.2%	10.1%
	TextCNN	ROUGE ↑	0.6393	0.6499	<u>0.7139</u>	0.6821	0.6532	0.5701	0.6393	<i>0.7001</i>	0.7340
		USE ↑	0.6901	0.7521	0.7024	0.6931	0.7410	0.7009	<u>0.8210</u>	<i>0.8162</i>	0.8709
		Mod ↓	15.9%	16.1%	<u>10.4%</u>	15.3%	13.4%	17.1 %	13.4%	17.2%	10.0%
IMDB	BERT-C	ROUGE ↑	0.7934	0.8123	0.8153	<i>0.8234</i>	0.8112	0.8233	<u>0.8531</u>	0.8021	0.8621
		USE ↑	0.8118	0.8561	0.8321	0.8246	<i>0.8691</i>	0.8369	<u>0.9080</u>	0.8600	0.9102
		Mod ↓	8.0%	6.7%	7.7%	6.7%	5.9%	8.3%	<u>6.2%</u>	9.2%	5.9%
	TextCNN	ROUGE ↑	0.8104	0.8321	0.8611	<i>0.8814</i>	0.8412	0.8600	<u>0.8821</u>	0.8123	0.9121
		USE ↑	0.8211	0.8905	0.8613	<i>0.7714</i>	<i>0.8911</i>	0.7979	<u>0.9000</u>	0.8600	0.9092
		Mod ↓	7.1%	7.1%	8.7%	<u>4.7%</u>	5.5%	6.3%	<i>5.2%</i>	7.6%	3.9%

The higher values of ROUGE and USE indicate better performance while the lower Mod indicates better performance. For each row, the best performance is highlighted in bold, the second best is highlighted in underline, and the third best is denoted with italic.

(a) Does our FBA method make more thrilling attacks to baselines?

We compare the attacking performance of the proposed FBA method and baselines in Table II. To be more specific, Table II demonstrates that FBA consistently outperforms other competing methods across different data domains, regardless of the structure of classifiers. It can thus be concluded that the proposed method FBA achieves the best-attacking performance, with the largest successful attack rate (SAR). We attribute such an outstanding attacking performance to the two prevailing

aspects of FBA. First, the proposed FBA could enlarge the searching domain by removing, substituting and inserting words compared with the strategies with only substitution such that FBA provides more possible attacking combinations. Second, FBA optimizes the performance by stochastically searching the domain. Most of the baselines perform a deterministic searching algorithm with Word Importance Rank (WIR) could get stuck in the local optima. Differently, such a stochastic mechanism helps skip the local optima and further maximize the attacking performance.

TABLE IV
QUALITY OF GENERATED ADVERSARIAL EXAMPLES MEASURED BY FLUENCY (PPL) AND GRAMMAR ERROR RATE (GERR) OF ATTACK ALGORITHMS

Dataset	Model	Metrics	Attack Methods								
			WMP	A2T	BAE	FAGA	BERT.A	CLARE	PWWS	PSO	FBA
AG's News	BERT-C	PPL ↓ GErr ↓	331 0.22	291 0.19	<u>142</u> 0.19	165 0.19	281 0.19	233 0.21	321 0.22	292 0.31	133 0.18
	TextCNN	PPL ↓ GErr ↓	281 0.23	199 0.17	<u>142</u> 0.19	145 0.15	241 0.15	213 0.19	277 0.20	143 <u>0.14</u>	133 0.13
Emotion	BERT-C	PPL ↓ GErr ↓	281 0.19	251 0.19	<u>233</u> 0.10	259 0.13	301 0.17	298 0.14	333 0.16	336 0.20	229 0.10
	TextCNN	PPL ↓ GErr ↓	288 0.16	268 0.11	<u>201</u> 0.10	241 0.13	221 0.13	256 0.16	299 0.19	301 0.18	182 0.10
SST2	BERT-C	PPL ↓ GErr ↓	213 0.20	211 0.18	<u>173</u> <u>0.15</u>	182 0.24	200 0.25	<u>155</u> 0.19	214 0.21	197 0.27	142 0.14
	TextCNN	PPL ↓ GErr ↓	198 0.19	164 0.21	<u>142</u> <u>0.14</u>	163 0.17	210 0.21	174 0.17	224 0.23	145 0.13	140 0.13
IMDB	BERT-C	PPL ↓ GErr ↓	91 0.28	<u>62</u> 0.22	70 <u>0.19</u>	90 0.21	83 <u>0.19</u>	101 0.18	88 <u>0.19</u>	90 0.22	60 0.18
	TextCNN	PPL ↓ GErr ↓	103 0.33	<u>69</u> 0.29	79 0.22	99 0.26	80 0.23	89 0.20	<u>69</u> 0.20	99 0.28	66 0.20

The lower values of PPL and GErr indicate better performance. For each row, the best quality is highlighted in bold, the second best quality is highlighted by underline, and the third best quality is denoted with italic.

TABLE V
ADVERSARIAL EXAMPLES OF EMOTION DATASET FOR VICTIM CLASSIFIER BERT-C

Attacks	Adversarial examples
A2T (Unsuccessful attack. True class score = 41.31%)	i spent wandering around still kinda dazed and not feeling sense particularly sociable social but because id been in hiding for a couple for days and it was getting to be a little unhealthy i made myself go down to the cross and hang out with folks
BAE (Unsuccessful attack. True class score = 33.25%)	i spent wandering around still kinda dazed and not feeling being particularly sociable but because id been in hiding for a couple for days and it was getting to be a little bit unhealthy i made myself go down to the cross and hang out with folks
FAGA (Successful attack . True class score = 13.32%)	i spent wandering around still kinda dazed and not feeling particularly sociable sympathetic but because id been in hiding for a couple few for days and it was getting to be a little unhealthy i made myself go down to the cross and hang out with folks
BERT.A (Unsuccessful attack. True class score = 77.04%)	i spent wandering around still kinda dazed and not feeling being particularly sociable happy but because id been in hiding for a couple for days and it was getting to be a little unhealthy i made myself go down to the cross and hang out with folks
CLARE (Successful attack . True class score = 10.54%)	i spent wandering around still kinda dazed and not feeling particularly sociable lonely but because id been in hiding for a couple for days and it was getting to be a little unhealthy i made myself go down to the cross and hang out with folks.
PWWS (Successful attack . True class score = 21.11%)	i spent wandering around still kinda dazed and not feeling palpate particularly sociable but because id been in hiding for a couple for days and it was getting to be a little unhealthy i made myself go down to the cross and hang out with folks
PSO (Successful attack . True class score = 6.90%)	i spent wandering around still kinda dazed and not feeling considering particularly sociable but because id been in hiding for a couple for days and it was getting to be a little unhealthy i made myself go down to the cross and hang out with folks dudes
FBA (Successful attack . True class score = 0.63%)	i spent wandering around still kinda dazed and not feeling sensing particularly sociable but because id been in hiding for a couple for days and it was getting to be a little unhealthy i made myself go down to the cross and hang out with folks

Blue texts are original words, while red ones are substitutions. The true class scores is placed inside the brackets. The lower true class score indicates better performance. The best attacks is bold. [42]

TABLE VI
ADVERSARIAL EXAMPLES OF SST2 DATASET FOR VICTIM CLASSIFIER TEXTCNN

Attacks	Adversarial examples
A2T (Successful attack. True class score = 21.31%)	an often-deadly boring short , strange reading of a classic whose witty dialogue is treated laced with a baffling casual approach
BAE (Unsuccessful attack. True class score = 32.10%)	an often-deadly boring , strange reading of a classic novel whose witty dialogue commentary is treated with a baffling casual approach
FAGA (Unsuccessful attack. True class score = 51.12%)	an often-deadly extremely boring , strange reading of a classic characters whose witty dialogue is treated with a baffling casual approach
BERT.A (Unsuccessful attack. True class score = 63.41%)	an often-deadly boring , strange critical reading of a classic whose witty spoken dialogue is treated with a baffling casual approach
CLARE (Unsuccessful attack. True class score = 10.54%)	an often-deadly boring , strange reading of a classic whose witty entire dialogue is treated with a baffling casual approach casualness
PWWS (Unsuccessful attack. True class score = 51.11%)	an often-deadly boring, strange casual reading of a classic whose witty dialogue is treated with a baffling more casual approach
PSO (Successful attack. True class score = 7.90%)	an often- deadly somewhat boring, strange humorous reading of a classic whose witty dialogue is treated with a baffling casual approach
FBA (Successful attack. True class score = 5.11%)	an often-deadly often-harmful boring, strange reading of a classic whose witty dialogue is treated with a baffling casual approach

Blue texts are original words, while red ones are substitutions. The true class scores is placed inside the brackets. The lower true class score indicates better performance. The best attacks is bold.

(b) *Is FBA superior to the baselines in terms of imperceptibility?*

We assess the imperceptibility of various attack strategies based on their semantic similarity (USE), modification rate (Mod), and word alignment (ROUGE) compared to the original text. Our findings, presented in Table III, show that the FBA approach is generally more effective than the baseline methods. While attacking BERT-C on AG's News, FBA performs similarly to PSO in terms of ROUGE, but maintains a higher semantic similarity (USE) on that dataset, suggesting that PSO's performance comes at the expense of imperceptibility. Hence, FBA outperforms the other methods in terms of imperceptibility. This is mainly due to the customized target distribution, which helps to avoid excessive modifications, and the use of both MLM and kNN to find more semantically similar substitutions.

(c) *Is the quality of adversarial examples generated by the FBA better than that crafted by the baselines?*

High-quality adversarial examples should be parsing-fluent and grammatically correct. From Table IV, we can find that FBA provides the lowest perplexity (PPL), which means the examples generated by FBA are more likely to appear in the corpus of evaluation. As our corpus is long enough and the evaluation model is broadly used, it indicates these examples are more likely to appear in natural language space, thus eventually leading to better fluency. For the grammar errors, the proposed method FBA is substantially better than the other baselines, which indicates better quality of the adversarial examples. We attribute such performance to our method of finding word substitution, constructing the candidates set by applying both MLM and kNN for synonym searching.

E. Ablation Studies

(a) *Evaluating the Effectiveness of MH*

To test the performance of the Metroplis-Hasting algorithm, we did an ablation study by making a comparison between FBA

and WMP whose adversarial candidates are not selected by the Metropolis-Hasting algorithm. Specifically, we perform these two attacks, FBA and WMP, to two classifiers, BERT-C and TextCNN pre-trained on dataset AG's News, and the experimental results are shown in Tables II, III, and IV. From these tables, FBA achieved better performance in both attack (SAR), imperceptibility (USE, Mod, ROUGE) and sentence quality (PPL, GErr), thus we can conclude that the Metropolis-Hasting algorithm is effective in selecting the adversarial candidates.

(b) *Evaluating the Impact of Actions: Removal, Insertion, and Substitution*

To evaluate the influence of different actions on attack performance, we set up six distinct pairs with associated probabilities, as outlined in Table VIII. In order to achieve equilibrium between the actions of removal and insertion, we consistently set their probabilities to be equal, represented as $P_{ins} = P_{rem}$.

The analysis of results from Table VIII can be categorized into three key facets: attack performance (SAR), imperceptibility (ROUGE, USE, Mod), and the quality of the generated examples (PPL, GErr). For attack performance, the group with $P_{sub} = 0.4$ exhibited the highest efficacy. Meanwhile, the groups with $P_{sub} = 0.6$ and $P_{sub} = 0.8$ trailed closely, differentiated by a slim margin. The initial three groups underperformed, primarily due to constraints that only consider examples with a USE exceeding 0.5 and a modification rate below 0.25. Hence, even though these groups could generate adversarial candidates capable of deceiving the target models, such examples are not regarded as successful adversarial instances. Furthermore, the group focusing solely on substitution also showcased a commendable success rate against overall performance.

Regarding imperceptibility, we observed an initial increase in performance with a rise in the substitution probability P_{sub} , which later began to decline. This trend can be attributed to the notion that a higher likelihood of insertions and removals can impact imperceptibility. Specifically, inserting or removing words may compromise language semantics, alignment,

TABLE VII
TARGETED ATTACK RESULTS ON EMOTION DATASET

Model	SAR $\uparrow$		ROUGE $\uparrow$		USE $\uparrow$		Mod $\downarrow$		PPL $\downarrow$		GErr $\downarrow$	
	PWWS	FBA	PWWS	FBA	PWWS	FBA	PWWS	FBA	PWWS	FBA	PWWS	FBA
BERT-C	21.23%	57.21%	0.4541	0.5101	0.6012	0.7732	13.1%	11.3%	341	299	0.28	0.22
TextCNN	32.61%	65.07%	0.5603	0.6198	0.6320	0.6511	21.3%	15.1%	411	223	0.29	0.28

The better-performed attack is highlighted in bold.

TABLE VIII
PERFORMANCE COMPARISON OF FBA ON THE AG NEWS DATASET AGAINST THE TEXTCNN VICTIM MODEL USING DIFFERENT ACTION PROBABILITIES

P_{sub}	P_{ins}	SAR	ROUGE	USE	Mod	PPL	GErr
0.0	0.5	69.12%	0.6711	0.7411	16.3%	231	0.21
0.2	0.4	78.10%	0.7011	0.7524	15.3%	201	0.19
0.4	0.3	94.12%	0.7911	0.7771	15.1%	179	0.17
0.6	0.2	<u>93.12%</u>	0.8711	0.8224	10.3%	133	<u>0.13</u>
0.8	0.1	<u>91.12%</u>	<u>0.8211</u>	0.8401	12.3%	112	<u>0.13</u>
1.0	0.0	87.01%	<u>0.8011</u>	<u>0.8333</u>	14.3%	110	0.12

The top three performances are highlighted in bold, underlined, and italics.

and parsing, as these actions can introduce significant losses or semantic redundancies. Simultaneously, if attackers focus exclusively on substitution, imperceptibility may suffer due to altering a larger set of words to bolster the attack's success. Hence, an optimal substitution probability likely exists that harmoniously balances imperceptibility. As for the quality of adversarial examples, there's a distinct pattern: the greater the substitution probability, the higher the quality. The experimental data suggests that inserting and removing affect sentence quality.

Our findings indicate that the performance is suboptimal when focusing solely on substitution or when excluding substitution altogether. This underscores the importance of considering all actions – substitution, removal, and insertion – to bolster the attack's effectiveness. It's imperative to gauge the overall success of adversarial attacks across three dimensions: attack potency, imperceptibility, and sentence quality. Engaging in adversarial attacks often necessitates trade-offs between imperceptibility and sentence quality, as documented in [42], [45]. Given varying attack objectives, attackers can adjust the substitution probability. Based on our experiments, a substitution probability of 0.6 (denoted as $P_{sub} = 0.6$) is recommended, as it strikes an ideal balance between attack efficacy and imperceptibility without undermining the textual quality.

(c) Evaluating the Effectiveness of Word Candidates Selection

Choosing the appropriate word candidates for substitution and insertion actions is crucial, as it directly impacts the success rate of attacks and imperceptibility. To evaluate the effectiveness of our word candidates selection method, we undertook an ablation study. This study compared performances utilizing a thesaurus (specifically, WordNet [18]), Masked Language Model (MLM), and Nearest Neighbors (NN) under L1, L2, and infinite norms. As shown from Table IX, our proposed method (MLM+ L_1 , MLM+ L_2 , MLM+ L_{inf}) for word candidate search exhibited superior performance than the baselines. We attribute this success to two main factors. First, WMP utilizes NN to identify

TABLE IX
PERFORMANCE METRICS FOR FBA AGAINST THE TEXTCNN MODEL ON THE AG NEWS DATASET USING VARIED WORD CANDIDATE SELECTION METHODS

Methods	SAR	ROUGE	USE	Mod	PPL	GErr
WordNet	55.12%	0.7103	0.7231	15.1%	260	0.21
L_{inf}	63.06%	0.7401	0.7010	14.8%	281	0.20
L_1	70.19%	0.7419	0.7533	14.1%	209	0.18
L_2	72.88%	0.7812	0.7695	14.0%	201	0.19
MLM	81.12%	0.7731	0.7031	15.3%	178	0.17
MLM+ L_{inf}	88.12%	0.8441	0.8001	13.3%	140	0.17
MLM+ L_1	92.42%	0.8713	0.8194	11.1%	136	0.14
MLM+ L_2	93.12%	<u>0.8711</u>	0.8224	10.3%	133	<u>0.13</u>

The best three performances for each metric are highlighted in bold, underline, and italics.

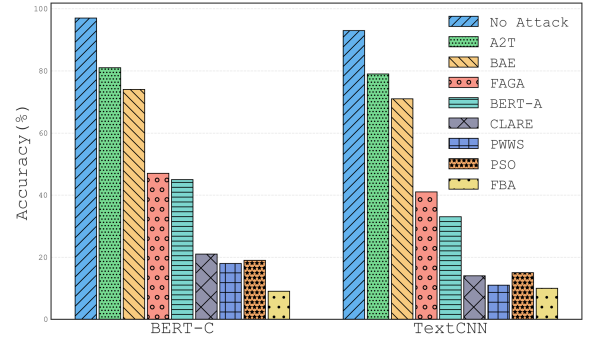


Fig. 1. Performance of transfer attacks to victim models (BERT-C and TextCNN) on Emotion. Lower accuracy of the victim models indicates higher transfer ability (the lower the better).

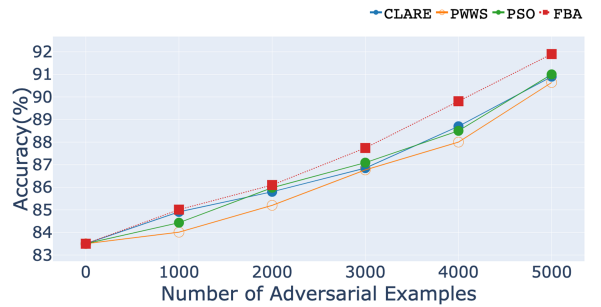


Fig. 2. Retraining accuracy of TextCNN with different numbers of adversarial examples included in the retraining. The higher the accuracy, the better the performance of the retraining.

‘potential’ synonyms that, although not always precise, capture the desired meaning. Second, the MLM is crucial in improving the sentence’s parsing structure. Furthermore, our observations indicate that the L2 norm marginally surpasses the L1 norm and significantly outperforms the infinite norm. While NN methods

TABLE X
TIME EFFICIENCY OF ATTACK ALGORITHMS EVALUATED WITH BERT-C ON THE EMOTION AND IMDB DATASET

Datasets	WMP	A2T	BAE	FAGA	BERT.A	CLARE	PWWS	PSO	FBA
Emotion	100.7	162.4	<u>21.7</u>	414.0	707.9	130.5	0.7	73.8	120.2
IMDB	155.7	431.4	<u>81.1</u>	781.0	1007.9	170.5	3.7	166.9	159.2

The metric of efficiency is second per example, which means a lower metric indicates a better efficiency.
The horizontally best 3 methods will be bold, underlined and italic.

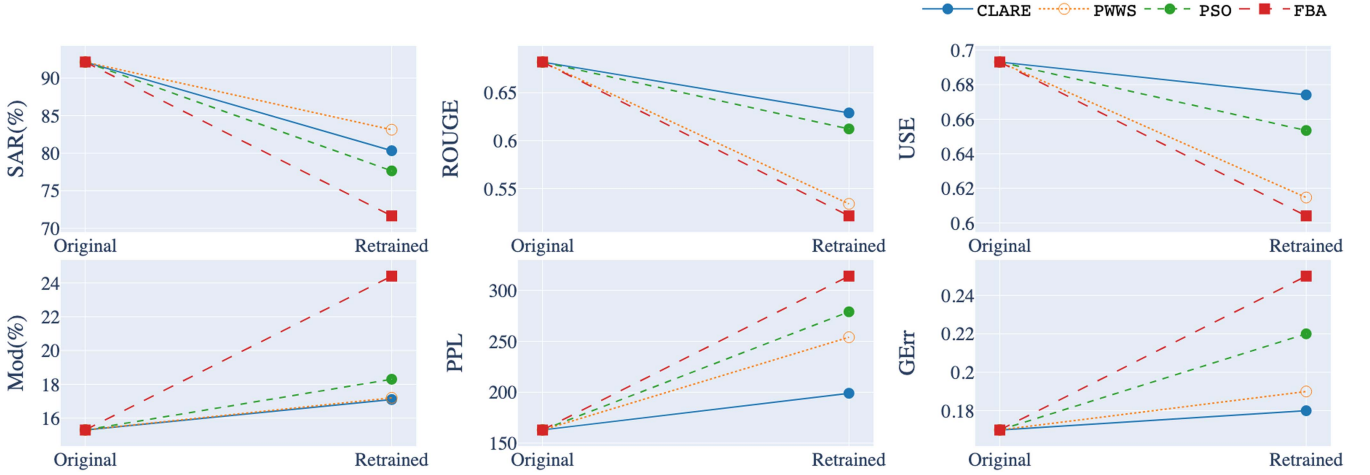


Fig. 3. We employ FAGA to attack the adversarial retrained TextCNNs which joins adversarial examples from different attacking strategies (CLARE, PWWS, PSO and FBA) to the training set of SST2. The lower metrics (SAR, ROUGE, USE) suggest a better performance in robustness while the higher metrics (Mod, PPL, GErr) suggest a better performance in robustness.

are not limited to any particular norm, our experimental results demonstrate the commendable efficacy of both L1 and L2 norms.

F. Transferability

The transferability of adversarial examples refers to the ability of an adversarial sample generated to deceive one model to deceive another model. This property is widely used as an important metric to evaluate the effectiveness of adversarial attacks. To assess the transferability of our method, we generate adversarial examples on BERT-C and TextCNN models and use them to attack the other model. The results of this evaluation are shown in Fig. 1, where the lower the classification accuracy, the higher the transferability. We observe that our method achieves the best transfer attack performance compared to other methods, indicating its ability to generate adversarial examples that can deceive different models. This property is particularly relevant in real-world scenarios where an attacker may not know the exact model deployed by the target and needs to generate adversarial examples that are effective against a range of models.

G. Targeted Attacks

A targeted adversarial attack aims to deceive the victim classifier by modifying the input sample in such a way that it is misclassified as a specified target class y' instead of its true class y . Compared to untargeted attacks, targeted attacks are considered more harmful as they provide the attacker with

more control over the final predicted label of the perturbed text. To conduct targeted attacks using FBA, we modify the definition of R in (19) by replacing $1 - F_y(x')$ with $F_{y'}(x')$. To evaluate the effectiveness of FBA for targeted attacks, we perform experiments on the Emotion dataset and report the results in Table VII. The results demonstrate that our proposed method achieves superior performance in terms of SAR, imperceptibility metrics (ROUGE, USE) and sentence quality metrics (GErr, PPL) for targeted attacks. This suggests that FBA can be effectively adapted for targeted adversarial attacks.

H. Adversarial Retraining

Adversarial retraining has emerged as a promising technique to improve the accuracy and robustness of machine learning models against adversarial attacks [46], [47], [48]. By incorporating adversarial examples generated by attack algorithms during the training process, models can learn to handle adversarial perturbations better and improve their robustness performance. This improvement is evident in Fig. 3, showcasing the benefits of adversarial retraining. Adversarial examples can be seen as informative features that reveal the weaknesses of the model, allowing it to learn more robust and discriminative features [49]. In order to assess the accuracy of retrained classifiers using adversarial examples, we added 1,000, 2,000, 3,000, 4,000, 5,000, and 6,000 randomly generated adversarial examples from the SST2 training set to the TextCNN model's training set. The

accuracy of the model on the clean test set was then compared after adversarial training and after appending varying numbers of adversarial examples. Based on the results presented in Fig. 2, we observed that the classifier trained using adversarial examples performed the best when the same number of adversarial examples were added from FBA.

I. Complexity and Qualitative Results

Experiments were run on a RHEL 7.9 system with an Intel(R) Xeon(R) Gold 6238R CPU (2.2 GHz, 28 cores - 26 enabled, 38.5 MB L3 Cache), an NVIDIA Quadro RTX 5000 GPU (3072 Cores, 384 Tensor Cores, 16 GB memory), and 88 GB RAM.

Table X presents the time taken to attack BERT and TextCNN classifiers on the Emotion dataset. Time efficiency is measured in seconds per example, where a lower value denotes better efficiency. As observed from Table X, while our WMP and FBA methods take longer than certain static baselines like PWWS and BAE, they outperform others such as CLARE, FAGA, A2T, and BA in terms of efficiency. It is noted that the extended run time of our methods compared to some baseline approaches suggests the additional time invested in seeking more optimal adversarial examples.

VI. CONCLUSION AND FUTURE WORK

The FBA algorithm uses the Word Manipulation Process to create many possible textual perturbations that can fool a classifier, making it effective for targeted attacks. Adversarial retraining can defend against attacks but is expensive and may reduce model accuracy. Current defence methods for text are not sufficient, so developing better defences is a promising future direction.

REFERENCES

- [1] X. Yang, W. Liu, J. Bailey, D. Tao, and W. Liu, "Bigram and unigram based text attack via adaptive monotonic heuristic search," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 706–714.
- [2] W. Wang, R. Wang, L. Wang, Z. Wang, and A. Ye, "Towards a robust deep neural network against adversarial texts: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 3159–3179, Mar. 2023.
- [3] J. Ebrahimi, A. Rao, D. Lowd, and D. Dou, "HotFlip: White-box adversarial examples for text classification," 2017, *arXiv:1712.06751*.
- [4] W. C. Gan and H. T. Ng, "Improving the robustness of question answering systems to question paraphrasing," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 6065–6075.
- [5] H. Sun, T. Zhu, Z. Zhang, D. Jin, P. Xiong, and W. Zhou, "Adversarial attacks against deep generative models on data: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3367–3388, Apr. 2023.
- [6] S. Gang and G. Ramakrishnan, "BAE: Bert-based adversarial examples for text classification," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2020, pp. 6174–6181.
- [7] Y. Zang et al., "Word-level textual adversarial attacking as combinatorial optimization," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 6066–6080.
- [8] L. Li, R. Ma, Q. Guo, X. Xue, and X. Qiu, "BERT-attack: Adversarial attack against bert using BERT," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2020, pp. 6193–6202.
- [9] M. Ni, C. Wang, T. Zhu, S. Yu, and W. Liu, "Attacking neural machine translations via hybrid attention learning," *Mach. Learn.*, vol. 111, pp. 3977–4002, 2022.
- [10] S. Ren, Y. Deng, K. He, and W. Che, "Generating natural language adversarial examples through probability weighted word saliency," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 1085–1097.
- [11] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2015, *arXiv:1412.6572*.
- [12] N. Papernot, P. D. McDaniel, A. Swami, and R. E. Harang, "Crafting adversarial input sequences for recurrent neural networks," 2016, *arXiv:1604.08275*.
- [13] M. Alzantot, Y. Sharma, A. Elgohary, B.-J. Ho, M. Srivastava, and K.-W. Chang, "Generating natural language adversarial examples," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2018, pp. 2890–2896.
- [14] R. Jia, A. Raghunathan, K. Göksel, and P. Liang, "Certified robustness to adversarial word substitutions," in *Proc. Conf. Empir. Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process.*, 2019, pp. 4129–4142.
- [15] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. Empir. Methods Natural Lang. Process.*, 2014, pp. 1532–1543.
- [16] T. Mikolov, K. Chen, G. S. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *Proc. Int. Conf. Learn. Representations*, 2013.
- [17] N. Mrkšić et al., "Counter-fitting word vectors to linguistic constraints," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2016, pp. 142–148.
- [18] G. A. Miller, R. Beckwith, C. Fellbaum, D. Gross, and K. J. Miller, "Introduction to WordNet: An on-line lexical database," *Int. J. Lexicogr.*, vol. 3, no. 4, pp. 235–244, 1990.
- [19] Z. Dong and Q. Dong, "HowNet-a hybrid language and knowledge resource," in *Proc. Int. Conf. Natural Lang. Process. Knowl. Eng.*, 2003, pp. 820–824.
- [20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Annu. Conf. North Amer. Chapter Assoc. Comput. Linguistics*, 2019, pp. 4171–4186.
- [21] Y. Liu et al., "RoBERTa: A robustly optimized bert pretraining approach," 2019, *arXiv:1907.11692*.
- [22] R. Rubinstein, "The cross-entropy method for combinatorial and continuous optimization," *Methodol. Comput. Appl. Probability*, vol. 1, no. 2, pp. 127–190, 1999.
- [23] X. Fan, B. Li, and S. Sisson, "Rectangular bounding process," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 7631–7641.
- [24] J. F. Haase et al., "A resource efficient approach for quantum and classical simulations of gauge theories in particle physics," *Quantum*, vol. 5, pp. 393–418, 2021.
- [25] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *J. Chem. Phys.*, vol. 21, no. 6, pp. 1087–1092, 1953.
- [26] W. K. Hastings, "Monte Carlo sampling methods using Markov chains and their applications," *Biometrika*, vol. 57, pp. 97–109, 1970. [Online]. Available: <https://api.semanticscholar.org/CorpusID:21204149>
- [27] K. Kumagai, I. Kobayashi, D. Mochihashi, H. Asoh, T. Nakamura, and T. Nagai, "Human-like natural language generation using monte carlo tree search," in *Proc. Workshop Comput. Creativity Natural Lang. Gener.*, 2016, pp. 11–18.
- [28] B. Harrison, C. Purdy, and M. O. Riedl, "Toward automated story generation with Markov chain Monte Carlo methods and deep neural networks," in *Proc. 13th Artif. Intell. Interactive Digit. Entertainment Conf.*, 2017, pp. 191–197.
- [29] X. Kang and F. Ren, "Sampling latent emotions and topics in a hierarchical Bayesian network," in *Proc. 7th Int. Conf. Natural Lang. Process. Knowl. Eng.*, 2011, pp. 37–42.
- [30] S. Singh, M. L. Wick, and A. McCallum, "Monte Carlo MCMC: Efficient inference by sampling factors," in *Proc. Joint Workshop Autom. Knowl. Base Construction Web-Scale Knowl. Extraction*, 2012, pp. 111–115.
- [31] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 8440–8451.
- [32] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, "MPNet: Masked and permuted pre-training for language understanding," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 16857–16867.
- [33] D. Li et al., "Contextualized perturbation for textual adversarial attack," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2021, pp. 5053–5069.
- [34] D. Jin, Z. Jin, J. T. Zhou, and P. Szolovits, "Is BERT really robust? A strong baseline for natural language attack on text classification and entailment," in *Proc. 34th AAAI Conf. Artif. Intell.*, 2020, pp. 8018–8025.

- [35] X. Wang, H. Jin, and X. Wang, "Natural language adversarial attacks and defenses in word level," 2019, *arXiv:1909.06723*. [Online]. Available: <https://api.semanticscholar.org/CorpusID:202577228>
- [36] G. A. Pavliotis, "Stochastic processes and applications," *Informe técnico*, Springer, pp. 30–44, 2015. [Online]. Available: <https://link.springer.com/book/10.1007/978-1-4939-1323-7>
- [37] D. P. Kroese, T. Taimre, and Z. I. Botev, *Handbook of Monte Carlo Methods*. Hoboken, NJ, USA: Wiley, 2011.
- [38] D. Cer et al., "Universal sentence encoder for English," in *Proc. Conf. Empir. Methods Natural Lang. Process. Syst. Demonstrations*, 2018, pp. 169–174.
- [39] X. Zhang, J. J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. 28th Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 649–657.
- [40] E. Saravia, H.-C. T. Liu, Y.-H. Huang, J. Wu, and Y.-S. Chen, "CARER: Contextualized affect representations for emotion recognition," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 3687–3697.
- [41] R. Socher et al., "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, Seattle, Washington, USA: Association for Computational Linguistics, 2013, pp. 1631–1642.
- [42] M. Ni, Z. Sun, and W. Liu, "Fraud's bargain attacks to textual classifiers via metropolis-hasting sampling (student abstract)," in *Proc. AAAI Conf. Artif. Intell.*, 2023, Art. no. 1936.
- [43] J. Y. Yoo and Y. Qi, "Towards improving adversarial training of NLP models," in *Proc. Findings Assoc. Comput. Linguistics*, 2021, pp. 945–956.
- [44] D. Naber et al., "A rule-based style and grammar checker," *Citeseer*, GRIN Verlag, 2003. [Online]. Available: <https://books.google.com.au/books?id=yKPswAEACAAJ>
- [45] P. Michel, X. Li, G. Neubig, and J. M. Pino, "On evaluation of adversarial perturbations for sequence-to-sequence models," 2019, *arXiv:1903.06620*.
- [46] H. Xia, S. Shao, C. Hu, R. Zhang, T. Qiu, and F. Xiao, "Robust clustering model based on attention mechanism and graph convolutional network," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 5203–5215, May 2023.
- [47] X. Ding, H. Fang, Z. Zhang, K.-K. R. Choo, and H. Jin, "Privacy-preserving feature extraction via adversarial training," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 4, pp. 1967–1979, Apr. 2022.
- [48] A. S. Chivukula, X. Yang, W. Liu, T. Zhu, and W. Zhou, "Game theoretical adversarial deep learning with variational adversaries," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 11, pp. 3568–3581, Nov. 2021.
- [49] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry, "Adversarial examples are not bugs, they are features," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, Art. no. 12.



Mingze Ni received the bachelor's degrees in science (Statistics and Mathematics) from Australian National University, and the bachelor of statistics (Honours) from the University of Queensland. He is currently working toward the PhD degree with the University of Technology Sydney. His researches focus on adversarial machine learning, textual attacks and natural language processing.



Zhensu Sun received the master's degree in software engineering from Tongji University, China. Currently, he is a research assistant with ShanghaiTech University, China. His research interest is the application of deep learning in software engineering to boost developers' productivity.



Wei Liu (Senior Member, IEEE) received the PhD degree in machine learning research from the University of Sydney, in 2011. He is currently the director of Future Intelligence Research Lab, and an Associate Professor in machine learning with the School of Computer Science, The University of Technology Sydney (UTS), Australia. Before joining UTS, he was a research fellow with the University of Melbourne and then a Machine Learning researcher with NICTA. His current research focuses are adversarial machine learning, game theory, causal inference, multimodal learning, and natural language processing. He has published more than 100 papers in top-tier journals and conferences. Additionally, he has won three Best Paper Awards and one Most Influential Paper Award.